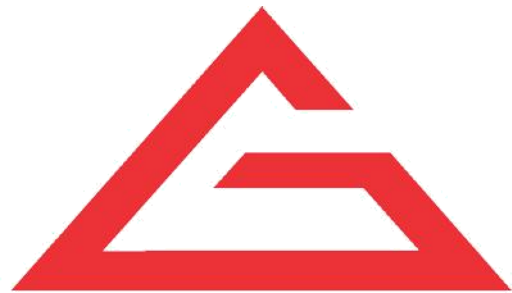


FLUTTER

LAB ASSIGNMENTS



G-TEC EDUCATION

— G-TEC Group of Institutions —

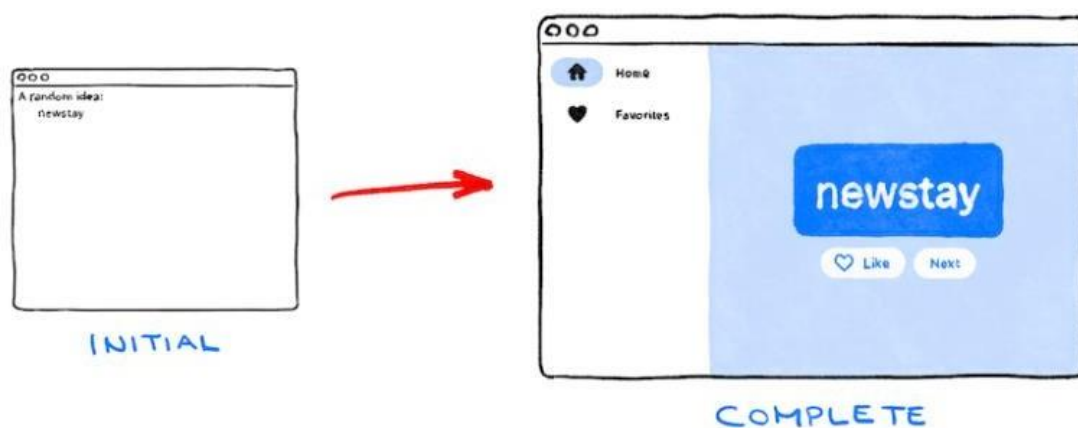
www.gteceducation.com

Admin Office House of G-TEC, Calicut-02., India. | Corp. Office Peace Centre, Singapore
— 228149

FLUTTER

Introduction

Flutter is Google's UI toolkit for building applications for mobile, web, and desktop from a single codebase. The application generates cool-sounding names, such as "newstay", "lightstream", "mainbrake", or "graypine". The user can ask for the next name, favorite the current one, and review the list of favorited names on a separate page. The app is responsive to different screen sizes.



Set Up your Flutter environment

To make this codelab as straightforward as possible, we assume you'll use **Visual Studio Code** (VS Code) as your development environment. It's free and works on all major platforms.

[file: downloadDownload Visual Studio Code](#)

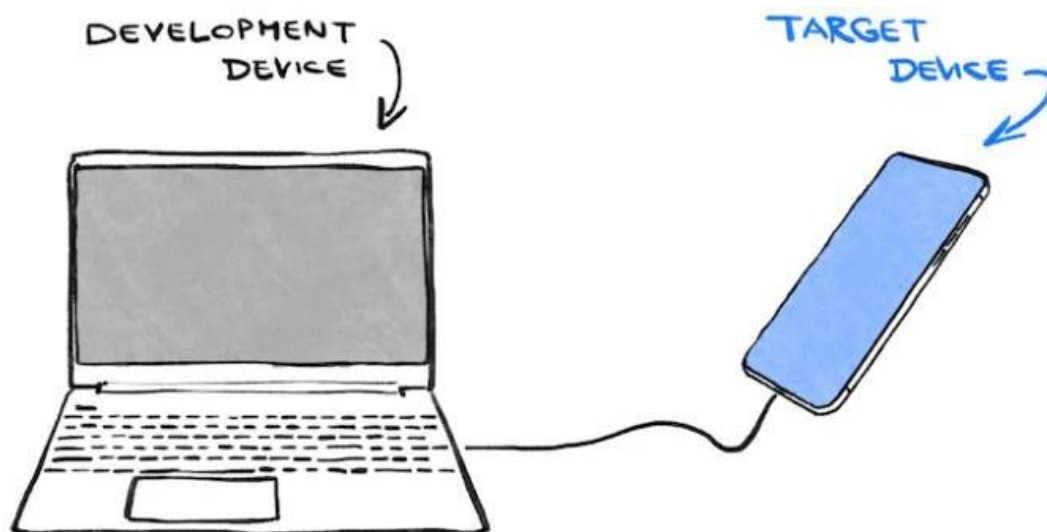
It's fine to use any editor you like: Android Studio, other IntelliJ IDEs, Emacs, Vim, or Notepad++. They all work with Flutter.

Choose a development target

Flutter is a multi-platform toolkit. Your app can run on any of the following operating systems:

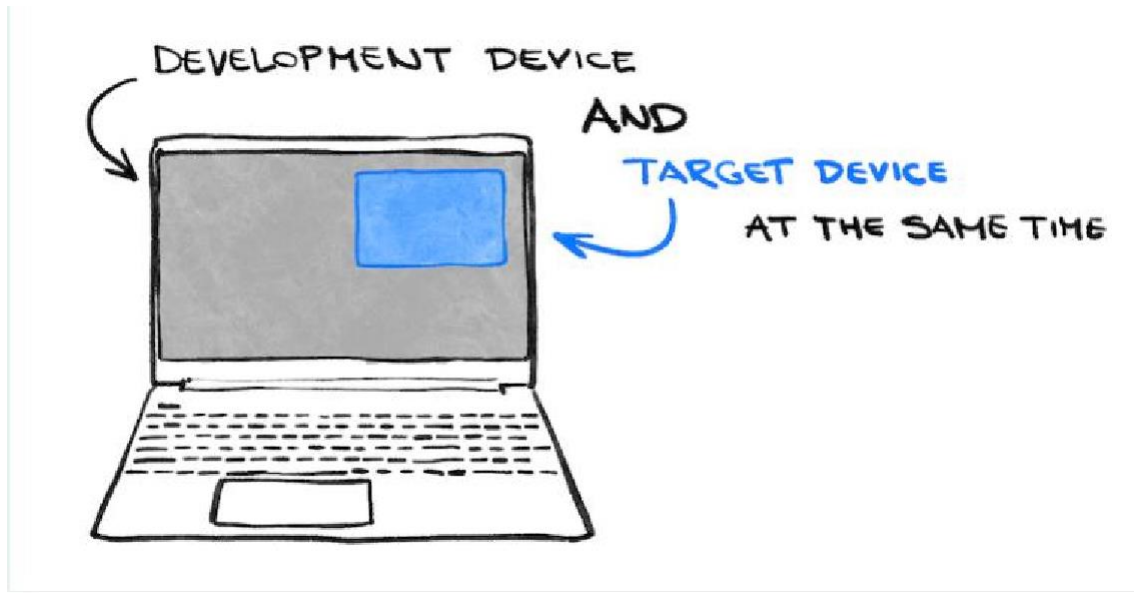
- iOS
- Android
- Windows
- macOS
- Linux
- web

However, it's common practice to choose a single operating system for which you will *primarily* develop. This is your "development target"—the operating system that your app runs on during development.



For example, say you're using a Windows laptop to develop a Flutter app. If you choose Android as your development target, you typically attach an Android device to your Windows laptop with a USB cable, and your app-in-development runs on that attached Android device. But you could also choose Windows as the development target, which means your app-in-development runs as a Windows app alongside your editor.

ip: We strongly recommend choosing your development device's Operating System as your development target. So, for example, if your computer runs Windows, choose Windows as the development target.



It might be tempting to select the web as your development target. The downside of this choice is that you lose one of Flutter's most useful development features: Stateful Hot Reload. Flutter can't hot-reload web applications.

Install Flutter

The most up-to-date instructions on how to install the Flutter SDK are always at docs.flutter.dev.

Install Flutter SDK

The instructions on the Flutter website cover not only the installation of the SDK itself, but also the development target-related tools and the editor plugins.

Remember that, for this codelab, you only need to install the following:

1. Flutter SDK
2. Visual Studio Code with the Flutter plugin
3. The software required by your chosen development target (for example: *Visual Studio* to target Windows, or *Xcode* to target macOS)

Create a Project

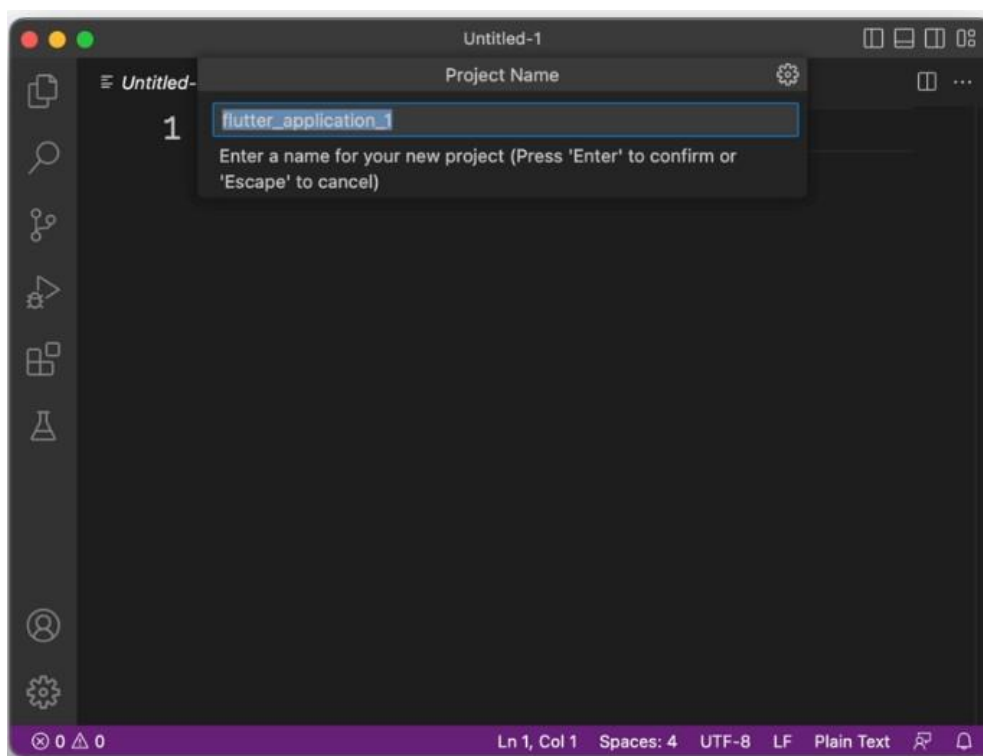
Create your first Flutter project

Launch Visual Studio Code and open the command palette

(with **F1** or **Ctrl+Shift+P** or **Shift+Cmd+P**). Start typing "flutter new". Select the **Flutter: New Project** command.

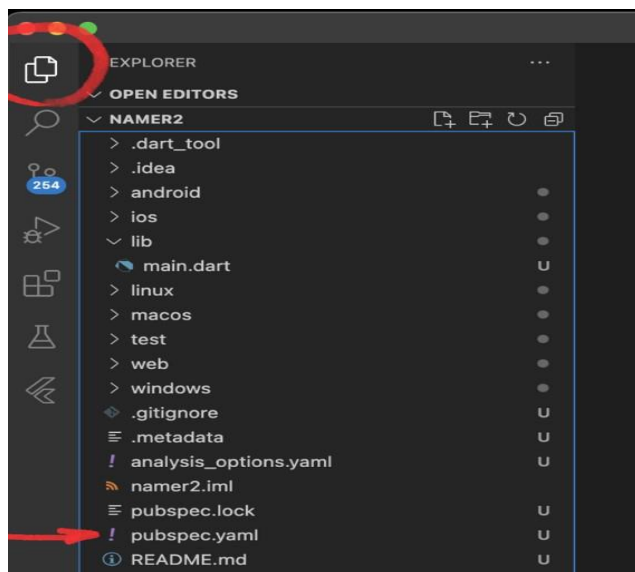
Next, select **Application** and then a folder in which to create your project. This could be your home directory, or something like `C:\src\`.

Finally, name your project. Something like `name_app` or `my_awesome_name`. (use only small letters to give your project name and give underscore to separate them)



Copy & Paste the initial app

In the left pane of VS Code, make sure that **Explorer** is selected, and open the **pubspec.yaml** file.



Replace the contents of this file with the following:

[pubspec.yaml](#)

name: namer_app

description: A new Flutter project.

publish_to: 'none' # Remove this line if you wish to publish to pub.dev

version: 0.0.1+1

environment:

sdk: ^3.1.1

dependencies:

flutter:

sdk: flutter

english_words: ^4.0.0

provider: ^6.0.0

dev_dependencies:

flutter_test:

sdk: flutter

flutter_lints: ^2.0.0

flutter:

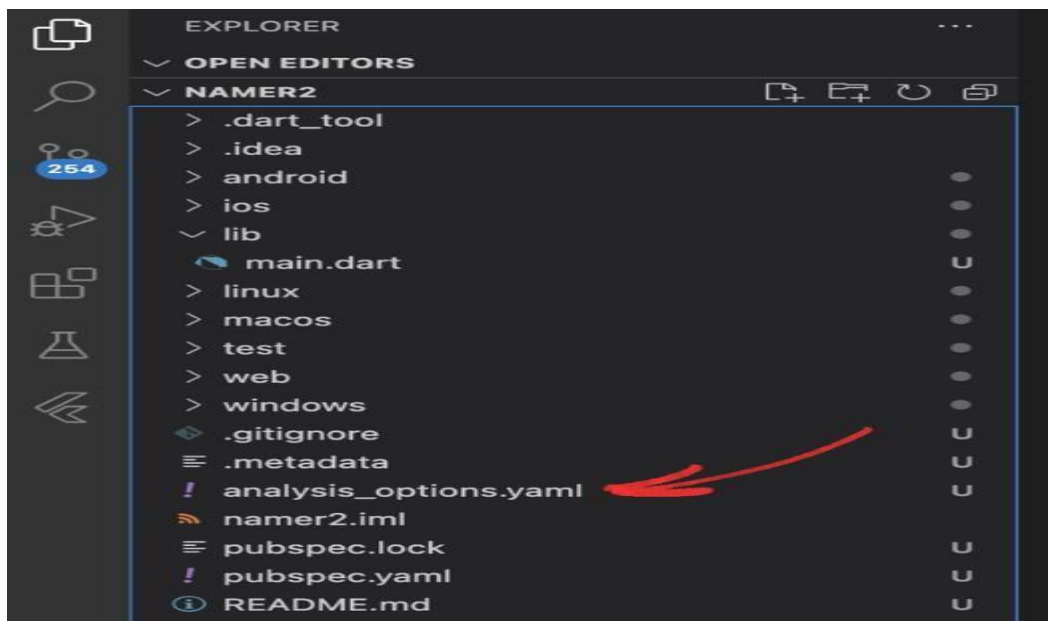
uses-material-design: true

The `pubspec.yaml` file specifies basic information about your app, such as its current version, its dependencies, and the assets with which it will ship.

Note: If you gave your app a name other than `namer_app`, you need to change the first line correspondingly.

Next, open another configuration file in the project, `analysis_options.yaml`.

Replace its contents with the following:



`analysis_options.yaml`

```
include: package:flutter_lints/flutter.yaml
```

```
linter:
```

```
rules:
```

```
avoid_print: false
```

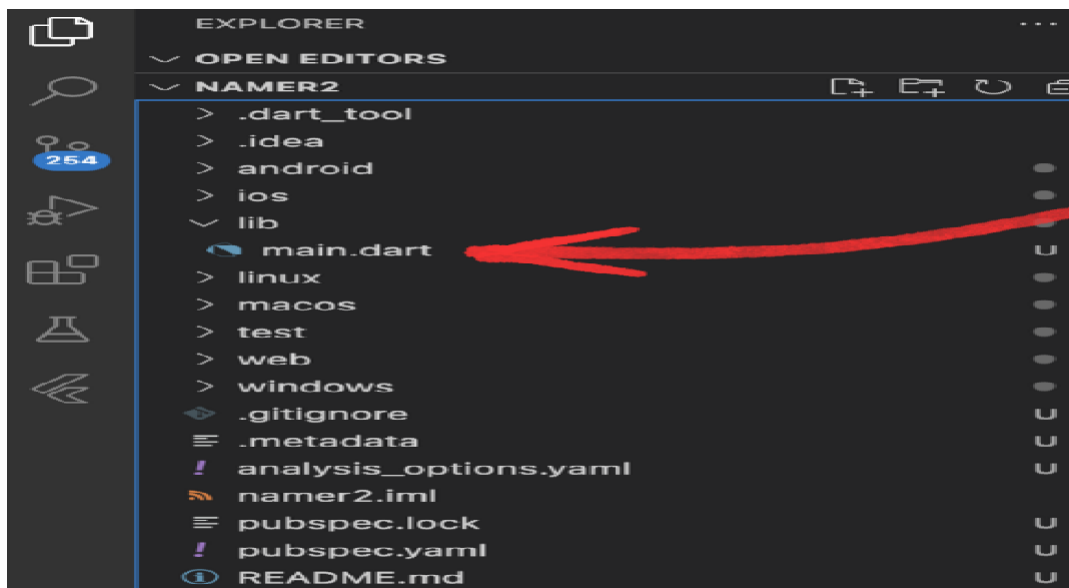
```
prefer_const_constructors_in_immutables: false
```



```
prefer_const_constructors: false
prefer_const_literals_to_create_immutables: false
prefer_final_fields: false
unnecessary_breaks: true
use_key_in_widget_constructors: false
```

his file determines how strict Flutter should be when analyzing your code. Since this is your first foray into Flutter, you're telling the analyzer to take it easy. You can always tune this later. In fact, as you get closer to publishing an actual production app, you will almost certainly want to make the analyzer stricter than this.

Finally, open the **main.dart** file under the **lib/** directory.



Replace the contents of this file with the following:

```
lib/main.dart
import 'package:english_words/english_words.dart';
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
void main() {
```

```
runApp(MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (context) => MyAppState(),
      child: MaterialApp(
        title: 'Namer App',
        theme: ThemeData(
          useMaterial3: true,
          colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepOrange),
        ),
        home: MyHomePage(),
      ),
    );
  }
}

class MyAppState extends ChangeNotifier {
  var current = WordPair.random();
}

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();

    return Scaffold(
      body: Column(
        children: [
          Text('A random idea:'),
          Text(appState.current.asLowerCase),
        ],
      ),
    );
  }
}
```

Add a Button

This step adds a **Next** button to generate a new word pairing.

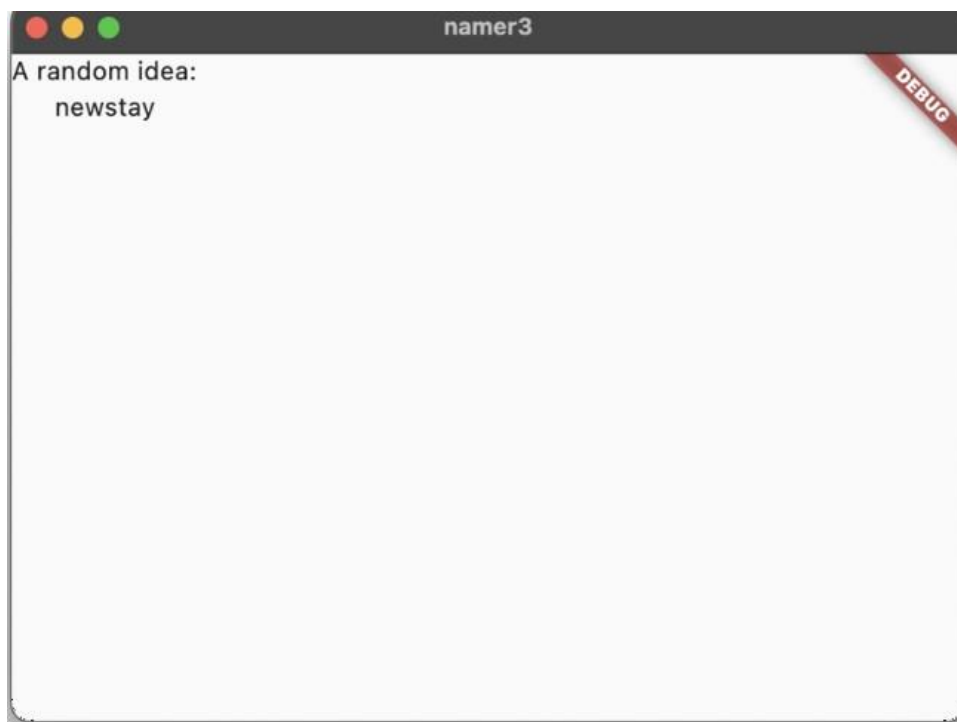
Launch the app

First, open `lib/main.dart` and make sure that you have your target device selected.

At the bottom right corner of VS Code, you'll find a button that shows the current target device. Click to change it.

While `lib/main.dart` is open, find the "play"  button in the upper right-hand corner of VS Code's window, and click it.

After about a minute, your app launches in debug mode. It doesn't look like much yet:



Note: If your development target is a mobile device, it's possible that the text is partly obscured by a notch or a status bar. You will learn how to fix this soon.

First Hot Reload

At the bottom of `lib/main.dart`, add something to the string in the first `Text` object, and save the file (with `Ctrl+S` or `Cmd+S`). For example:

lib/main.dart

```
// ...

return Scaffold(
  body: Column(
    children: [
      Text('A random AWESOME idea:'), // ← Example change.
      Text(appState.current.asLowerCase),
    ],
  ),
);

// ...
```

Notice how the app changes immediately but the random word stays the same. This is Flutter's famous *Siāīeful Hoī Reload* at work. Hot reload is triggered when you save changes to a source file.

Adding a button

Next, add a button at the bottom of the `Column`, right below the second `Text` instance.

[lib/main.dart](#)

```
// ...

return Scaffold(body:
  Column(children: [
    Text('A random AWESOME idea:'),
    Text(appState.current.asLowerCase),

    // ↓ Add this.
    ElevatedButton(
      onPressed: () {
        print('button pressed!');
      },
      child: Text('Next'),
    ),

  ],
```

```
),  
);  
// ...
```

When you save the change, the app updates again: A button appears and, when you click it, the *Debug Console* in VS Code shows a **button pressed!** message.

As much fun as it is to watch the *Debug Console*, you want the button to do something more meaningful. Before getting to that, though, take a closer look at the code in `lib/main.dart`, to understand how it works.

[lib/main.dart](#)

```
// ...  
  
void main() {  
  runApp(MyApp());  
}  
  
// ...
```

At the very top of the file, you'll find the `main()` function. In its current form, it only tells Flutter to run the app defined in `MyApp`.

[lib/main.dart](#)

```
// ...
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return ChangeNotifierProvider(  
      create: (context) => MyAppState(),
```

```
      child: MaterialApp(  
        title: 'Namer App',
```

```
        theme: ThemeData(  
          useMaterial3: true,
```

```
          colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepOrange),  
        ),
```

```
        home: MyHomePage(),  
      ),
```

```
    );  
  }
```

```
}
```

```
}
```

```
// ...
```

The MyApp class extends StatelessWidget. Widgets are the elements from which you build every Flutter app. As you can see, even the app itself is a widget.

Note: We'll get to the explanation of StatelessWidget (versus StatefulWidget) later.

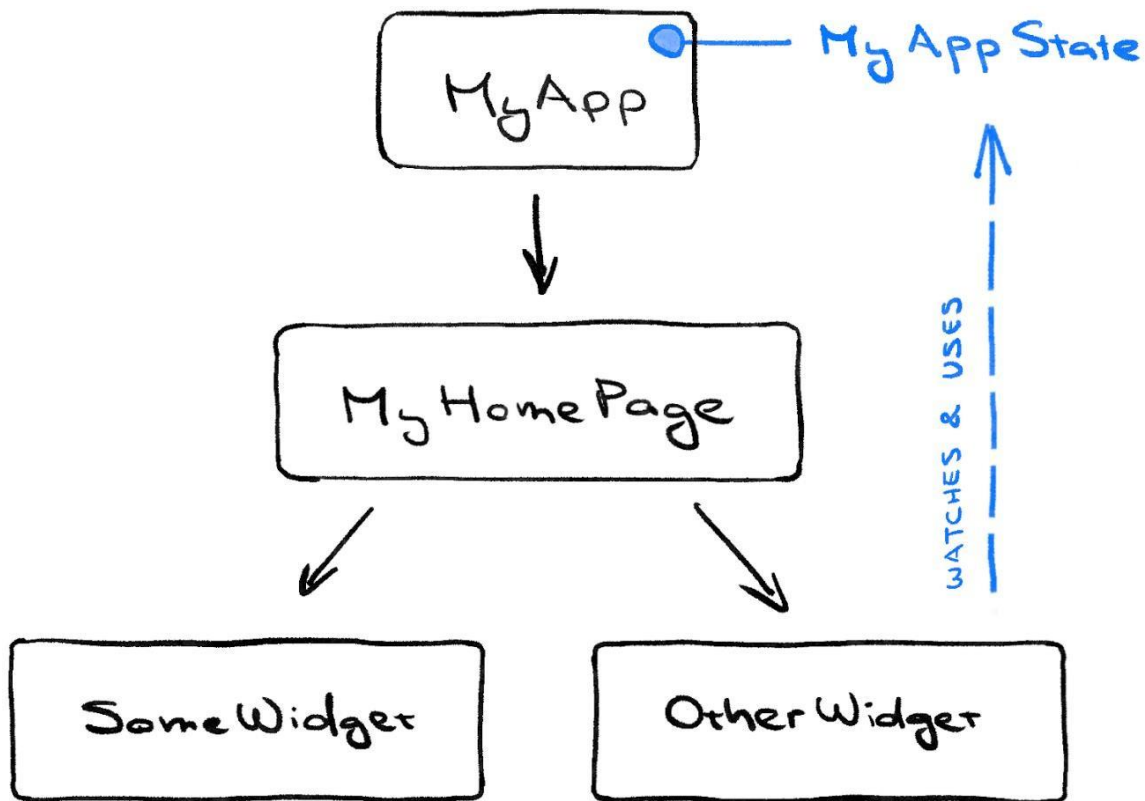
The code in **MyApp** sets up the whole app. It creates the app-wide state (more on this later), names the app, defines the visual theme, and sets the "home" widget—the starting point of your app.

[lib/main.dart](#)

```
// ...  
  
class MyAppState extends ChangeNotifier {  
  var current = WordPair.random();  
}  
  
// ...
```

Next, the **MyAppState** class defines the app's...well...state. This is your first foray into Flutter, so this codelab will keep it simple and focused. Here are many powerful ways to manage app state in Flutter. One of the easiest to explain is **ChangeNotifier**, the approach taken by this app.

- **MyAppState** defines the data the app needs to function. Right now, it only contains a single variable with the current random word pair. You will add to this later.
- The state class extends **ChangeNotifier**, which means that it can *notify* others about its own *changes*. For example, if the current word pair changes, some widgets in the app need to know.
- The state is created and provided to the whole app using a **ChangeNotifierProvider** (see code above in **MyApp**). This allows any widget in the app to get hold of the state.



[lib/main.dart](#)

// ...

```
class MyHomePage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) { // ← 1  
    var appState = context.watch<MyAppState>(); // ← 2  
  
    return Scaffold( // ← 3  
      body: Column( // ← 4  
        children: [  
          Text('A random AWESOME idea:'), // ← 5  
          Text(appState.current.asLowerCase), // ← 6  
          ElevatedButton(  
            onPressed: () {  
              print('button pressed!');  
            },  
            child: Text('Next'),  
          ),  
        ],  
      ),  
    );  
  }  
}
```



```
    ),  
    ],          // ← 7  
  ),  
  );  
}  
}  
  
// ...
```

Lastly, there's **MyHomePage**, the widget you've already modified. Each numbered line below maps to a line-number comment in the code above:

1. Every widget defines a **build()** method that's automatically called every time the widget's circumstances change so that the widget is always up to date.
2. **MyHomePage** tracks changes to the app's current state using the **watch** method.
3. Every **build** method must return a widget or (more typically) a nested *tree* of widgets. In this case, the top-level widget is **Scaffold**. You aren't going to work with **Scaffold** in this code lab, but it's a helpful widget and is found in the vast majority of real-world Flutter apps.
4. **Column** is one of the most basic layout widgets in Flutter. It takes any number of children and puts them in a column from top to bottom. By default, the column visually places its children at the top. You'll soon change this so that the column is centered.
5. You changed this **Text** widget in the first step.
6. This second **Text** widget takes **appState**, and accesses the only member of that class, **current** (which is a **WordPair**). **WordPair** provides several helpful getters, such as **asPascalCase** or **asSnakeCase**. Here, we use **asLowerCase** but you can change this now if you prefer one of the alternatives.
7. Notice how Flutter code makes heavy use of trailing commas. This particular comma doesn't need to be here, because **children** is the last (and also *only*) member of this particular **Column** parameter list. Yet it is generally a good idea to use trailing commas: they make adding more members trivial, and they also serve as a hint for Dart's auto-formatter to put a newline there. For more information, see [Code formatting](#).

Next, you'll connect the button to the state.

Your first behavior

Scroll to **MyAppState** and add a **getNext** method.

[lib/main.dart](#)

```
// ...  
  
class MyAppState extends ChangeNotifier {  
  var current = WordPair.random();  
  
  // ↓ Add this.  
  void getNext() {  
    current = WordPair.random();  
    notifyListeners();  
  }  
}  
  
// ...
```

The new **getNext()** method reassigns **current** with a new random **WordPair**.

It also calls **notifyListeners()** (a method of **ChangeNotifier**) that ensures that anyone watching **MyAppState** is notified.

All that remains is to call the **getNext** method from the button's callback.

[lib/main.dart](#)

```
// ...  
  
ElevatedButton(  
  onPressed: () {  
    appState.getNext(); // ← This instead of print().  
  },  
  child: Text('Next'),  
),  
  
// ...
```

Save and try the app now. It should generate a new random word pair every time you press the **Next** button.